[BUG] AttributeError: 'JacobianTape' object has no attribute 'grad_method

https://github.com/PennyLaneAI/pennylane/issues/2268

ROW 22

[BUG] AttributeError: 'JacobianTape' object has no attribute 'grad_method' When using param shift gradient tape and sampled gradients #2268

New issue

Closed

DuckTigger opened on Mar 3, 2022

Expected behavior

Using `qml.tape.QubitParamShiftTape()` as the gradient tape I expect to get results for the gradients of parameters.

Actual behavior

The code runs into an `AttributeError` from line 118 of `pennylane/tape/qubit_param_shift.py`:
`AttributeError: 'JacobianTape' object has no attribute 'grad_method'`

Line 118 is calling `op.grad_method` where `op` is an instance of `JacobianTape`

Additional information

Reproducible at all times.

Source code

```
import tensorflow as tf
import cirq
import pennylane as qml

qubits = cirq.GridQubit.rect(2, 2)
n_qubits = 4
shots = 1000
device = qml.device('cirq.simulator', wires=n_qubits, shots=shots)


def ansatz(val, params):
    qml.BasisEmbedding(val, list(range(n_qubits)))
    for i, w in enumerate(range(n_qubits)):
        qml.RX(params[i], wires=w)


def loss_fn(Y, y_pred):
    y_pred = tf.reshape(y_pred, (shots, n_qubits))
    return tf.reduce_mean(tf.subtract(Y, y_pred))


def test_gen():
    while True:
        yield [0 for _ in range(len(qubits))], [0 for _ in range(len(qubits))]


def create_model():
    @qml.qnode(device, interface='tf', diff_method='parameter-shift')
    def model(inputs, params):
        ansatz(inputs, params)
        measurement = [qml.sample(qml.PauliZ(w)) for w in range(len(qubits))]
        return measurement
    return model


def train():
    tf.keras.backend.set_floatx('float64')
    model = create_model()
    weight_shapes = {"params": n_qubits}
    qlayer = qml.qnn.KerasLayer(model, weight_shapes, output_dim=n_qubits)
    model = tf.keras.models.Sequential([qlayer])
    for epoch in range(10):
        X, Y = next(test_gen())
        with qml.tape.QubitParamShiftTape() as tape:
            pred_y = model(X, training=True)
            cost_val = loss_fn(Y, pred_y)
        grads = tape.jacobian(device, model.trainable_weights[0])
```

Assignees
No one assigned

Labels
bug

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
        grads = tape.jacobian(device, model.trainable_weights[0])


if __name__ == '__main__':
    train()
```

Tracebacks

```
Traceback (most recent call last):
  File "/home/andrew/.config/JetBrains/PyCharm2021.3/scratches/scratch.py", line 50, in <module>
    train()
  File "/home/andrew/.config/JetBrains/PyCharm2021.3/scratches/scratch.py", line 46, in train
    grads = tape.jacobian(device, model.trainable_weights[0])
  File "/home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages/pennylane/tape/qubi
    return super().jacobian(device, params, **options)
  File "/home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages/pennylane/tape/jacc
    diff_methods = self._grad_method_validation(method)
  File "/home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages/pennylane/tape/jacc
    self._update_gradient_info()
  File "/home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages/pennylane/tape/jacc
    info["grad_method"] = self._grad_method(i, use_graph=True)
  File "/home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages/pennylane/tape/qubi
    if op.grad_method == "F":
AttributeError: 'JacobianTape' object has no attribute 'grad_method'
```

System information

```
WARNING: pip is being invoked by an old script wrapper. This will fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead of running pip directly.
Name: PennyLane
Version: 0.21.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author: None
Author-email: None
License: Apache License 2.0
Location: /home/andrew/Documents/PhD/non-linearities/non-lin-env/lib/python3.8/site-packages
Requires: appdirs, pennylane-lightning, cachetools, scipy, numpy, toml, retworkx, autograd, autoray, semantic-
Required-by: PennyLane-Lightning, PennyLane-Cirq
Platform info:           Linux-5.16.9-zen1-1-zen-x86_64-with-glibc2.34
Python version:          3.8.12
Numpy version:           1.19.5
Scipy version:           1.7.0
Installed devices:
- default.gaussian (PennyLane-0.18.0)
- default.mixed (PennyLane-0.18.0)
- default.qubit (PennyLane-0.18.0)
- default.qubit.autograd (PennyLane-0.18.0)
- default.qubit.jax (PennyLane-0.18.0)
- default.qubit.tf (PennyLane-0.18.0)
- default.qubit.torch (PennyLane-0.18.0)
- default.tensor (PennyLane-0.18.0)
- default.tensor.tf (PennyLane-0.18.0)
- lightning.qubit (PennyLane-Lightning-0.18.0)
- cirq.mixedsimulator (PennyLane-Cirq-0.17.1)
- cirq.pasqal (PennyLane-Cirq-0.17.1)
- cirq.qsim (PennyLane-Cirq-0.17.1)
- cirq.qsimh (PennyLane-Cirq-0.17.1)
- cirq.simulator (PennyLane-Cirq-0.17.1)
```

Existing GitHub issues

- [x] I have searched existing GitHub issues to make sure the issue does not already exist.

DuckTigger added bug on Mar 3, 2022

josh146 on Mar 3, 2022 — Member

Hi @DuckTigger! In the `train()` function, as you are computing the gradients of a Keras model, I believe you want the `tf.GradientTape()` context manager.

`qml.tape.QubitParamShiftTape` is an internal part of PennyLane unrelated to TensorFlow gradients, that is due to soon be removed from PennyLane 🙂

Below I've modified your functions to take this into account, which will allow the gradients to be returned:

```
def loss_fn(Y, y_pred):
    return tf.reduce_mean(Y - y_pred)

def test_gen():
    return tf.zeros([2, len(qubits)], dtype=tf.float64)

@qml.qnode(device, interface='tf', diff_method='parameter-shift')
def circuit(inputs, params):
    qml.BasisEmbedding(inputs, list(range(n_qubits)))

    for i, w in enumerate(range(n_qubits)):
        qml.RX(params[i], wires=w)

    return [qml.expval(qml.PauliZ(w)) for w in range(len(qubits))]

def train():
    tf.keras.backend.set_floatx('float64')
    weight_shapes = {"params": n_qubits}

    qlayer = qml.qnn.KerasLayer(circuit, weight_shapes, output_dim=n_qubits)
    model = tf.keras.models.Sequential([qlayer])

    for epoch in range(10):
        X, Y = test_gen()

        with tf.GradientTape() as tape:
            pred_y = model(X, training=True)
            cost_val = loss_fn(Y, pred_y)

        grad = tape.gradient(cost_val, model.trainable_weights[0])

    print(cost_val, grad)
```

DuckTigger on Mar 3, 2022 — Author

Hi Josh, thanks for the speedy reply!

I moved to using ParamShift from tensorflow gradients as I'm trying to use sampling in my model - this is fundamental, so I can't use expval.

When I try this code with sampling I get `gradients == None`, which is why I moved to using `QubitParamShiftTape` (and pennylane) in the first place.

Being a keras model is not fundamental to me, but sampling is.

I get the same issue when removing all references to tensorflow:

```
import cirq
import pennylane as qml
import numpy as np

qubits = cirq.GridQubit.rect(2, 2)
n_qubits = 4
shots = 1000
device = qml.device('cirq.simulator', wires=n_qubits, shots=shots)


def ansatz(val, params):
    qml.BasisEmbedding(val, list(range(n_qubits)))
    for i, w in enumerate(range(n_qubits)):
```

I get the same issue when removing all references to tensorflow:

```
import cirq
import pennylane as qml
import numpy as np

qubits = cirq.GridQubit.rect(2, 2)
n_qubits = 4
shots = 1000
device = qml.device('cirq.simulator', wires=n_qubits, shots=shots)


def ansatz(val, params):
    qml.BasisEmbedding(val, list(range(n_qubits)))
    for i, w in enumerate(range(n_qubits)):
        qml.RX(params[i], wires=w)


def loss_fn(Y, y_pred):
    y_pred = np.reshape(y_pred, (shots, n_qubits))
    return np.mean(Y - y_pred)


def test_gen():
    while True:
        yield [0 for _ in range(len(qubits))], [0 for _ in range(len(qubits))]


def create_model():
    @qml.qnode(device, diff_method='parameter-shift')
    def model(inputs, params):
        ansatz(inputs, params)
        measurement = [qml.sample(qml.PauliZ(w)) for w in range(len(qubits))]
        return measurement
    return model


def train():
    model = create_model()
    initial_params = np.random.normal(size=(n_qubits,))
    for epoch in range(10):
        X, Y = next(test_gen())
        with qml.tape.QubitParamShiftTape() as tape:
            pred_y = model(X, initial_params)
            cost_val = loss_fn(Y, pred_y)
        grads = tape.jacobian(device, initial_params)
        print(grads)


if __name__ == '__main__':
    train()
```

josh146 on Mar 3, 2022 Member

> Being a keras model is not fundamental to me, but sampling is.

In that case **@DuckTigger** you can use *single shot expectation values* 🙂 These are expectation values computed with a 'single shot', so they:

- are identical to samples
- support differentiation, unlike samples

To do this, you can set

```
shots=[(1, 1000)]
```

This means 'use 1000 shots, 1 shot per expectation'. I tested this single change in the code in my comment above, and it seems to work well.

```
        with qml.tape.QubitParamShiftTape() as tape:
            pred_y = model(X, initial_params)
            cost_val = loss_fn(Y, pred_y)
        grads = tape.jacobian(device, initial_params)
        print(grads)


if __name__ == '__main__':
    train()
```

josh146 on Mar 3, 2022 — Member

> Being a keras model is not fundamental to me, but sampling is.

In that case **@DuckTigger** you can use *single shot expectation values* 🙂 These are expectation values computed with a 'single shot', so they:

- are identical to samples
- support differentiation, unlike samples

To do this, you can set

```
shots=[(1, 1000)]
```

This means 'use 1000 shots, 1 shot per expectation'. I tested this single change in the code in my comment above, and it seems to work well.

DuckTigger on Mar 3, 2022 — Author

Just checked and it works -
Amazing, thanks very much for your time!

DuckTigger closed this as completed on Mar 3, 2022